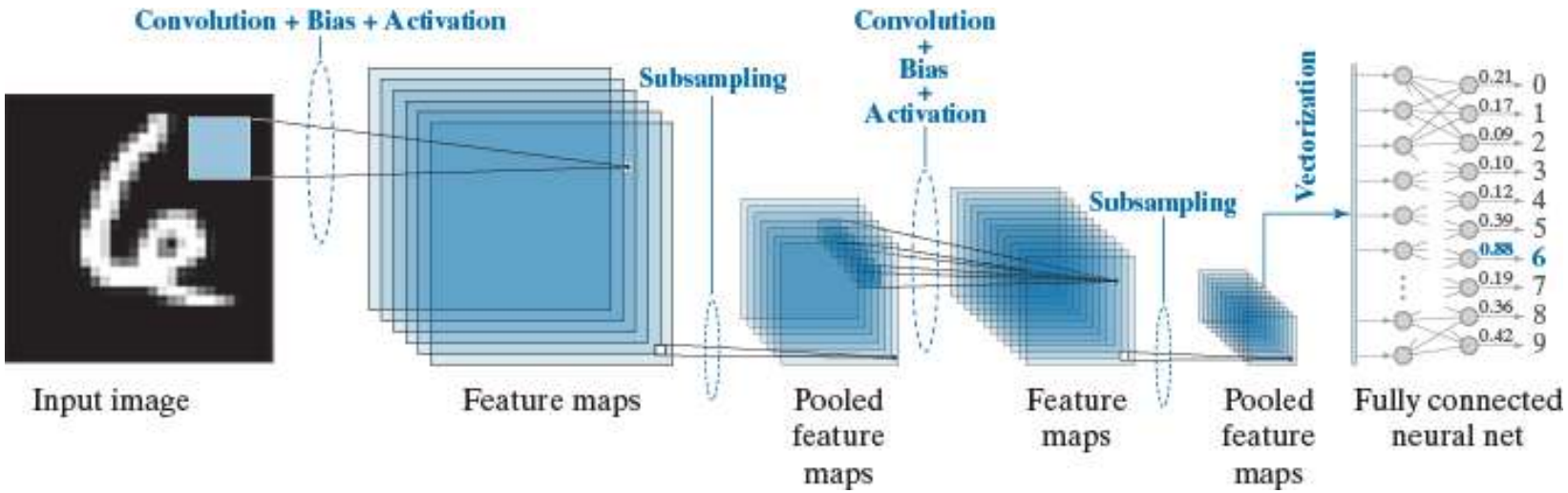
Image Processing

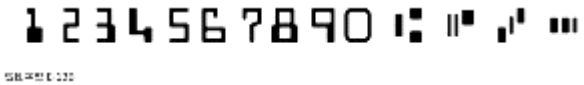


12.5 Neural networks and deep learning

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Image Pattern Classification

- Classification by/based on
  - Prototype matching

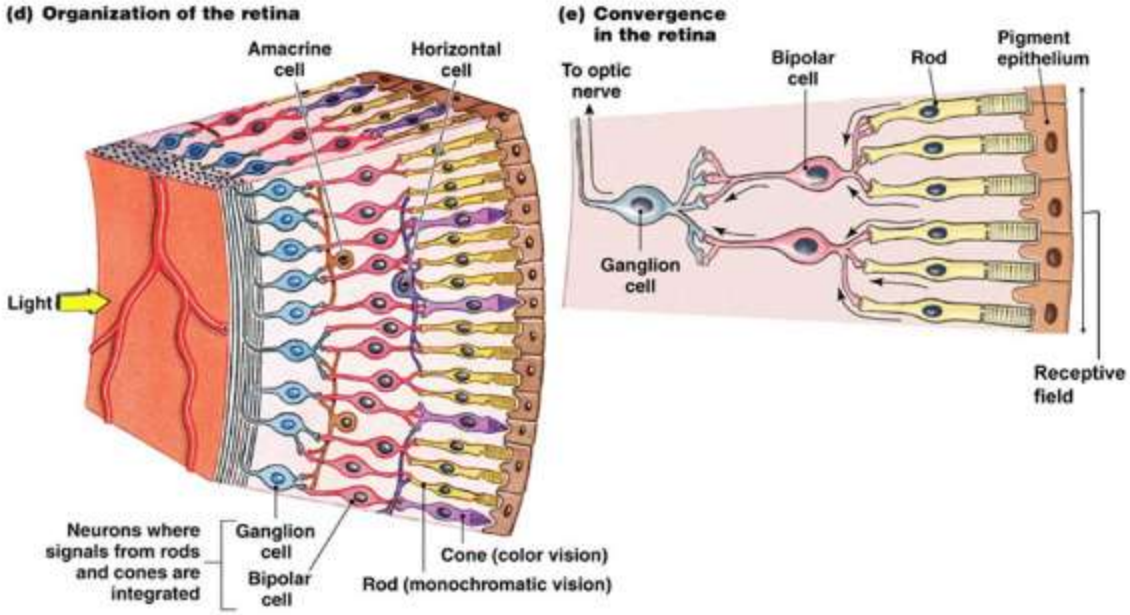


    - Make the features so unique and easily detectable that classification is simple
  - Optimal statistical formulation
    - Selecting parameters that yield optimal classification performance in a statistical sense
  - Neural networks
    - Usually raw data can be used, without the need for engineered features
      - Fully connected neural networks
      - Convolutional neural networks

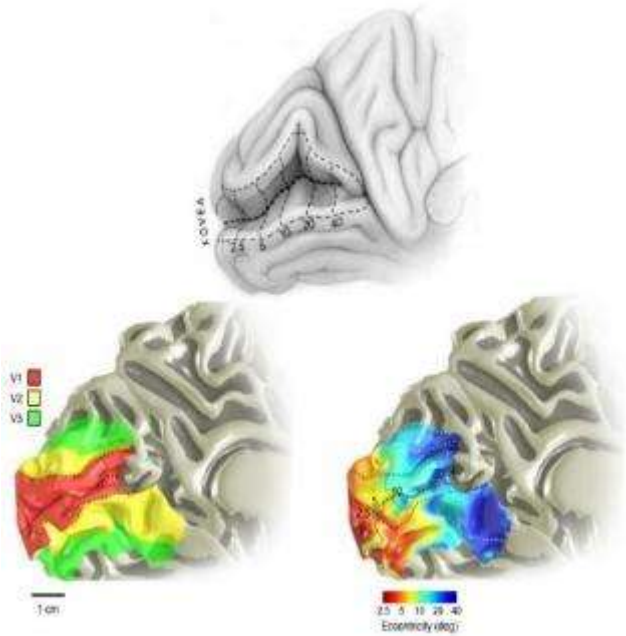
為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Artificial Neurons

- **Nonlinear computing elements, organized as networks similar in some respects to the visual cortex of mammals**
  - Neural networks, neurocomputers, parallel distributed processing models, neuromorphic systems, layered self-adaptive networks, connectionist models



Copyright © 2007 Pearson Education, Inc., publishing as Benjamin Cummings.



為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Perceptron

- **A single perceptron unit learns a linear boundary between two linearly separable pattern classes**

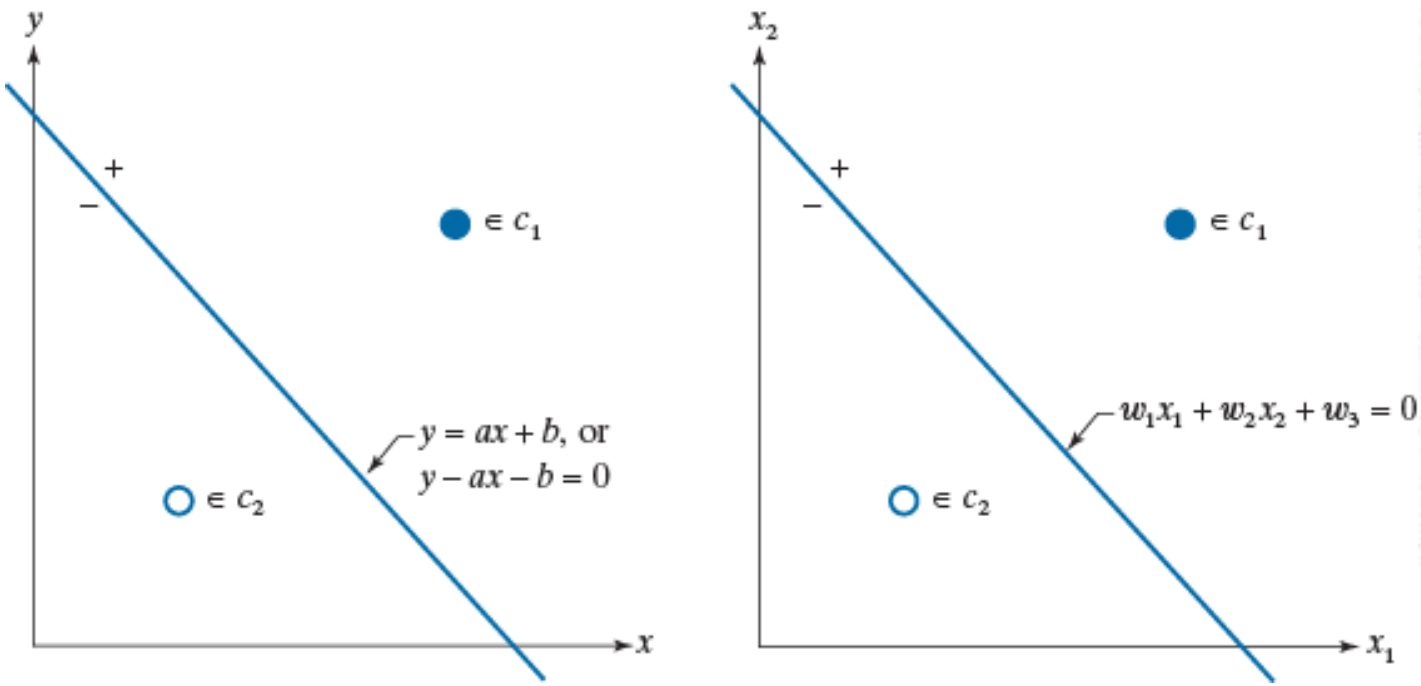


FIGURE 12.22 (a) The simplest two-class example in 2-D, showing one possible decision boundary out of an infinite number of such boundaries. (b) Same as (a), but with the decision boundary expressed using more general notation.

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Perceptron: Hyperplane

- **For a point in *n* dimensions, the test would be against a hyperplane**

$$w_1x_1 + w_2x_2 + \cdots + w_nx_n + w_{n+1} = 0$$

- In summation form:

$$\sum_{i=1}^{n} w_ix_i + w_{n+1} = 0$$

- In vector form:

$$\boldsymbol{w}^T\boldsymbol{x} + \omega_{n+1} = 0$$

weight vector bias

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Perceptron for Class Separation

- **Class separation problem:**
  - To find a set of weights with the property

$$\boldsymbol{w}^T\boldsymbol{x}+\omega_{n+1} = \begin{cases} > 0 \text{ if } \boldsymbol{x} \in c_1 \\ < 0 \text{ if } \boldsymbol{x} \in c_2 \end{cases}$$

- **The perceptron is an implementation of an algorithm to find a solution**
  - By iteratively stepping through the patterns of each of two classes
  - It converges in a finite number of iterations if the classes are linearly separable

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Perceptron Training Algorithm

- **Let w(1) be a vector with arbitrary values, $w_{n+1}(1)$ be an arbitrary value, and a learning rate α**
- **Do the following for k=2, 3, …**
  - For a pattern vector, x(k), at step k:

$$\boldsymbol{w}^T\boldsymbol{x}+\omega_{n+1} = \begin{cases} > 0 \text{ if } \boldsymbol{x} \in c_1 \\ < 0 \text{ if } \boldsymbol{x} \in c_2 \end{cases}$$

1) If $\mathbf{x}(k) \in c_1$ and $\boldsymbol{w}^T(k)\mathbf{x}(k) + w_{n+1}(k) \le 0$, let

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) + \alpha\mathbf{x}(k)$$
$$\omega_{n+1}(k+1) = \omega_{n+1}(k) + \alpha$$

2) If $\mathbf{x}(k) \in c_2$ and $\boldsymbol{w}^T(k)\mathbf{x}(k) + w_{n+1}(k) \ge 0$, let

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) - \alpha\mathbf{x}(k)$$
$$\omega_{n+1}(k+1) = \omega_{n+1}(k) - \alpha$$

3) Otherwise, let

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k)$$
$$\omega_{n+1}(k+1) = \omega_{n+1}(k)$$

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Perceptron Training Algorithm

- **To simplify the notion:**

define $\mathbf{x} \triangleq [x_1, x_2, \ldots, x_n, 1]^T$ and $\boldsymbol{w} \triangleq [w_1, w_2, \ldots, w_n, w_{n+1}]^T$.

$$\boldsymbol{w}^T\mathbf{x} = \begin{cases} > 0 & \text{if } \mathbf{x} \in c_1 \\ < 0 & \text{if } \mathbf{x} \in c_2 \end{cases}$$

$$\boxed{\sum_{i=1}^{n} w_i x_i + w_{n+1} = 0}$$

1′) If $\mathbf{x}(k) \in c_1$ and $\boldsymbol{w}^T(k)\mathbf{x}(k) \le 0$, let

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) + \alpha\mathbf{x}(k)$$

2′) If $\mathbf{x}(k) \in c_2$ and $\boldsymbol{w}^T(k)\mathbf{x}(k) \ge 0$, let

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) - \alpha\mathbf{x}(k)$$

3′) Otherwise, let

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k)$$

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Perceptron for Class Separation

- **Perceptron operations:**
  - Sum of products
  - Activation function: a thresholding function

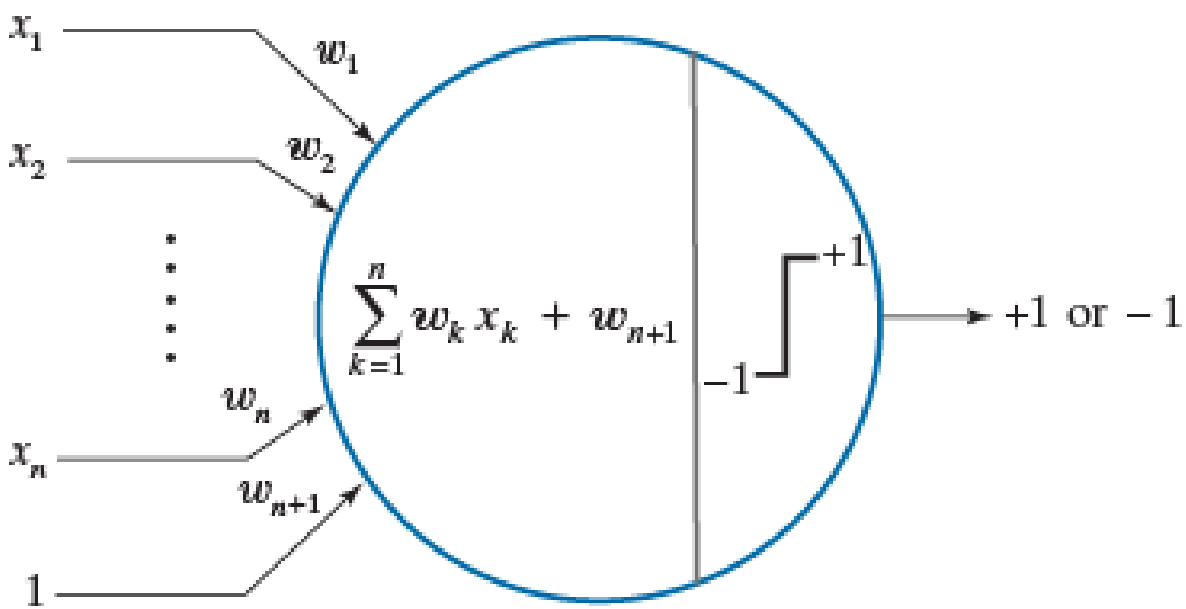


FIGURE 12.23 Schematic of a perceptron, showing the operations it performs.

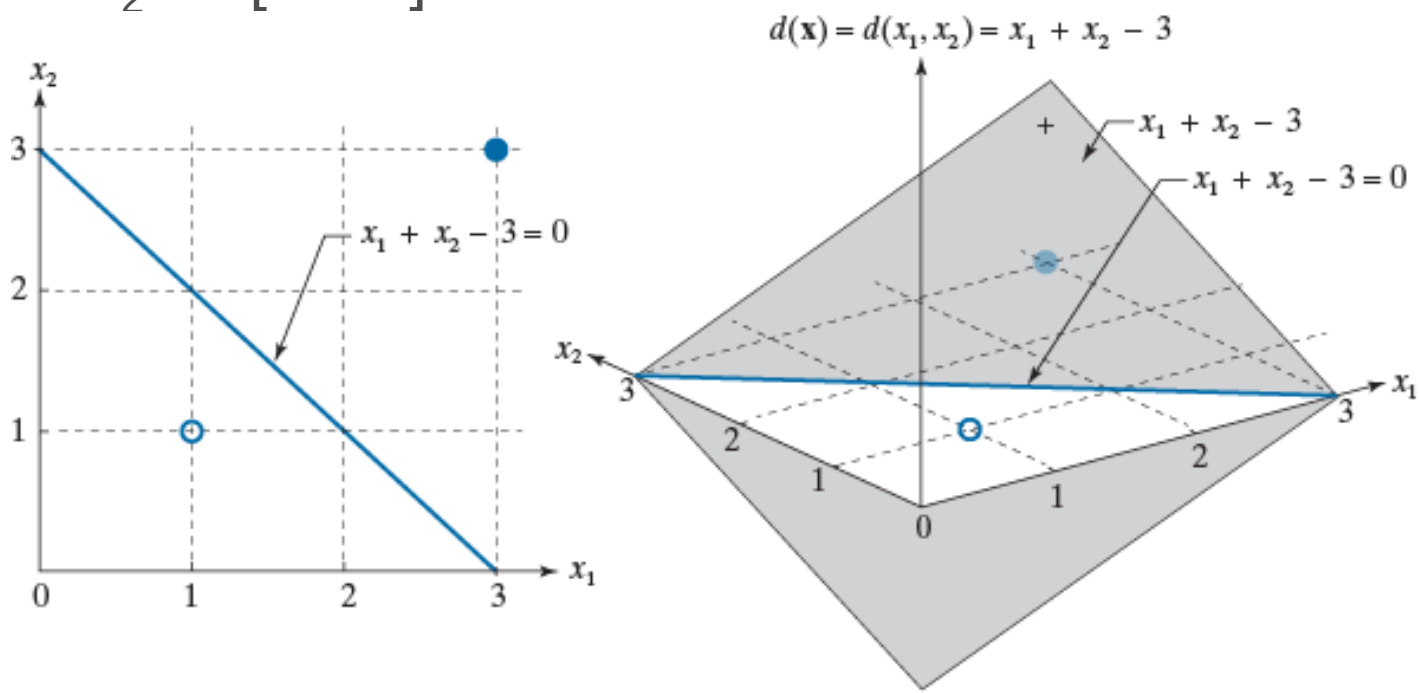
為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Perceptron for Class Separation

- **Example:**
  - Class c_1: $x=[3\ 3\ 1]^T$, and
  - Class c_2: $x=[1\ 1\ 1]^T$

a b

FIGURE 12.24 (a) Segment of the decision boundary learned by the perceptron algorithm. (b) Section of the decision surface. The decision boundary is the intersection of the decision surface with the x_1x_2-plane.



為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Perceptron for Class Separation

- **Epoch: one complete iteration through all patterns of the training**
- **Start with $w(1)=[0\ 0\ 0]^T$, $\alpha=1$**

For $k=1$, $\mathbf{x}(1)=[3\ 3\ 1]^t \in c_1$, and $\boldsymbol{w}(1)=[0\ 0\ 0]^t$. Their inner product is zero,

$$\boldsymbol{w}^T(1)\mathbf{x}(1)=\begin{bmatrix}0 & 0 & 0\end{bmatrix}\begin{bmatrix}3\\3\\1\end{bmatrix}=0$$

so Step 1′ of the second version of the training algorithm applies:

$$\boldsymbol{w}(2)=\boldsymbol{w}(1)+\alpha\mathbf{x}(1)=\begin{bmatrix}0\\0\\0\end{bmatrix}+(1)\begin{bmatrix}3\\3\\1\end{bmatrix}=\begin{bmatrix}3\\3\\1\end{bmatrix}$$

For $k=2$, $\mathbf{x}(2)=[1\ 1\ 1]^T \in c_2$ and $\boldsymbol{w}(2)=[3\ 3\ 1]^T$. Their inner product is

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Perceptron for Class Separation

$$\boldsymbol{w}^T(2)\mathbf{x}(2) = \begin{bmatrix} 3 & 3 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = 7$$

The result is positive when it should have been negative, so Step 2′ applies:

$$\boldsymbol{w}(3) = \boldsymbol{w}(2) - \alpha\mathbf{x}(2) = \begin{bmatrix} 3 \\ 3 \\ 1 \end{bmatrix} - (1)\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \\ 0 \end{bmatrix}$$

We have gone through a complete training epoch with at least one correction, so we cycle through the training set again.

For $k = 3$, $\mathbf{x}(3) = [3\ 3\ 1]^T \in c_1$, and $\boldsymbol{w}(3) = [2\ 2\ 0]^T$. Their inner product is positive (i.e., 6) as it should be because $\mathbf{x}(3) \in c_1$. Therefore, Step 3′ applies and the weight vector is not changed:

$$\boldsymbol{w}(4) = \boldsymbol{w}(3) = \begin{bmatrix} 2 \\ 2 \\ 0 \end{bmatrix}$$

For $k = 4$, $\mathbf{x}(4) = [1\ 1\ 1]^T \in c_2$, and $\boldsymbol{w}(4) = [2\ 2\ 0]^T$. Their inner product is positive (i.e., 4) and it should have been negative, so Step 2′ applies:

$$\boldsymbol{w}(5) = \boldsymbol{w}(4) - \alpha\mathbf{x}(4) = \begin{bmatrix} 2 \\ 2 \\ 0 \end{bmatrix} - (1)\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix}$$

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Perceptron for Class Separation

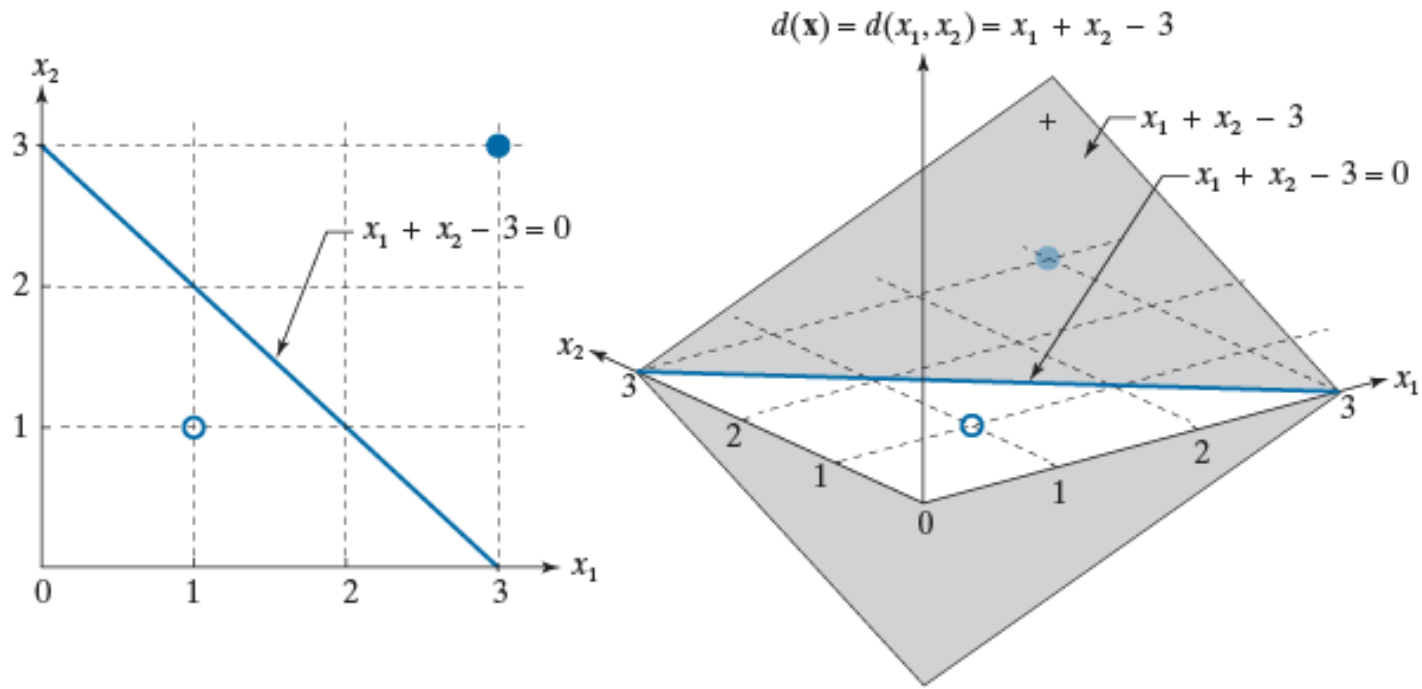
- ➔ $$\boldsymbol{w} = \boldsymbol{w}(12) = \begin{bmatrix} 1 \\ 1 \\ -3 \end{bmatrix}$$

Decision boundary $x_1 + x_2 - 3 = 0$

Decision surface $d(\mathbf{x}) = d(x_1, x_2) = x_1 + x_2 - 3,$

a b

FIGURE 12.24 (a) Segment of the decision boundary learned by the perceptron algorithm. (b) Section of the decision surface. The decision boundary is the intersection of the decision surface with the x_1x_2-plane.



為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

LMSE Algorithm

- **What happens for nonseparable pattern classes?**
- **We can minimize the error between the actual and desired responses at any training step**
  - The error function should be differentiable and have a unique minimum
  - For example:

$$E(\boldsymbol{w}) = \frac{1}{2}\left(r - \boldsymbol{w}^T \mathbf{x}\right)^2$$

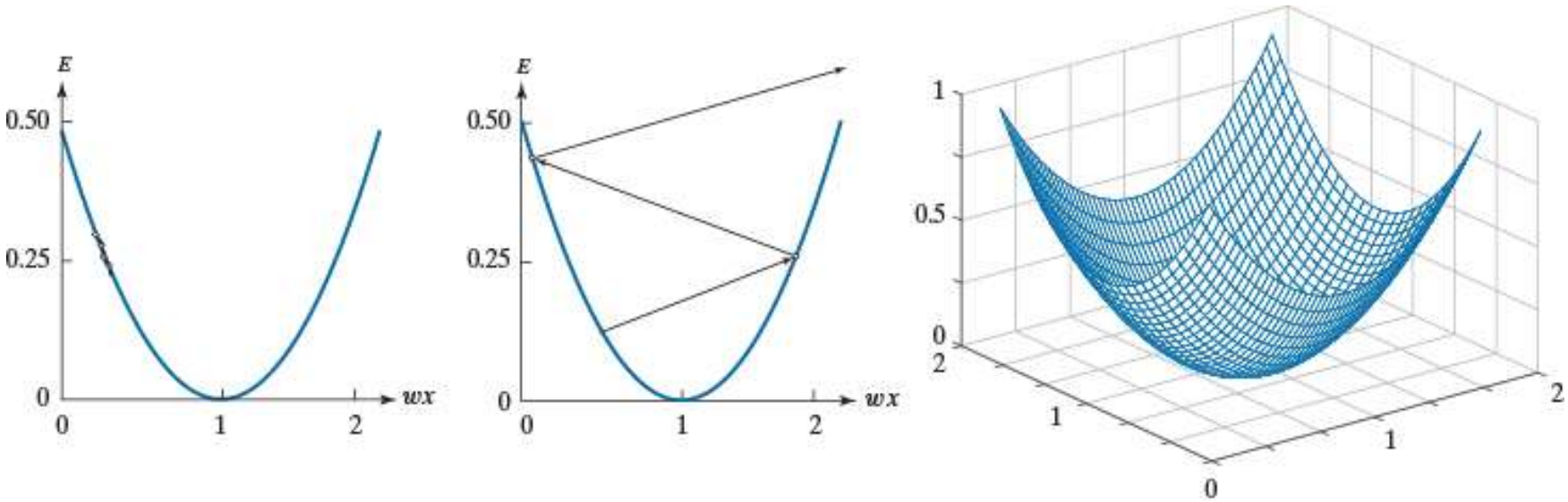
  - Find minimum of E(w) using an iterative gradient descent algorithm

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) - \alpha\left[\frac{\partial E(\boldsymbol{w})}{\partial \boldsymbol{w}}\right]_{\boldsymbol{w}=\boldsymbol{w}(k)}$$

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

LMSE Algorithm

❖ Iterative gradient descent algorithm



a b c

FIGURE 12.25 Plots of E as a function of wx for $r = 1$. (a) A value of α that is too small can slow down convergence. (b) If α is too large, large oscillations or divergence may occur. (c) Shape of the error function in 2-D.

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) - \alpha\left[\frac{\partial E(\boldsymbol{w})}{\partial \boldsymbol{w}}\right]_{\boldsymbol{w}=\boldsymbol{w}(k)}$$

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

LMSE Algorithm

- **What happens for nonseparable pattern classes?**
  - Find minimum of E(w) using an iterative gradient descent algorithm

$$E(\boldsymbol{w}) = \frac{1}{2}\left(r - \boldsymbol{w}^T\mathbf{x}\right)^2 \qquad \frac{\partial E(\boldsymbol{w})}{\partial \boldsymbol{w}} = -\left(r - \boldsymbol{w}^T\mathbf{x}\right)\mathbf{x}$$

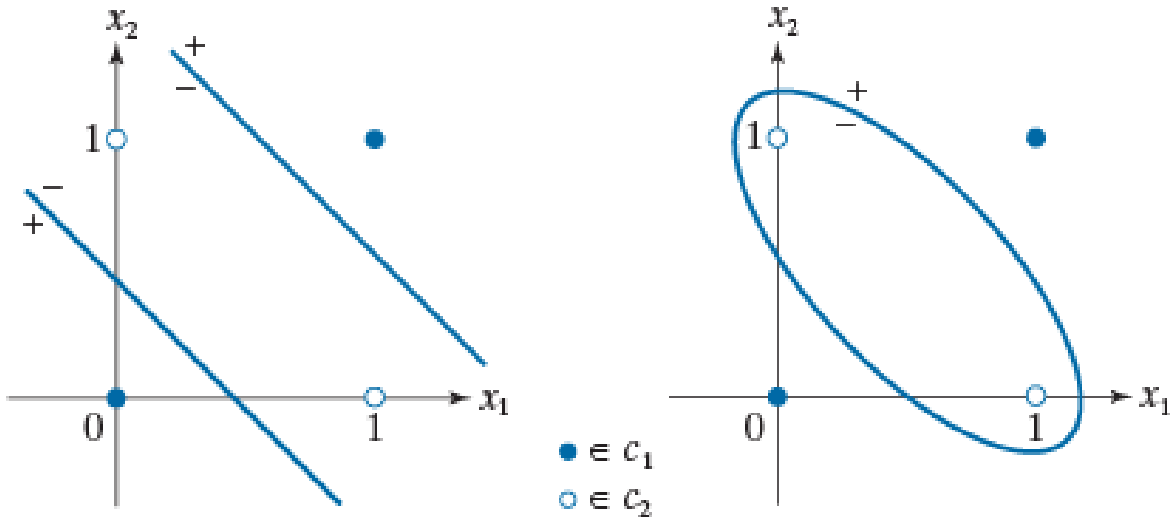
$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) + \alpha\left[r(k) - \boldsymbol{w}^T(k)\mathbf{x}(k)\right]\mathbf{x}(k)$$

  - The solution at convergence may not be a hyperplane that fully partitions two linearly separable classes

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Limitation of Single Linear Decision Boundary

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

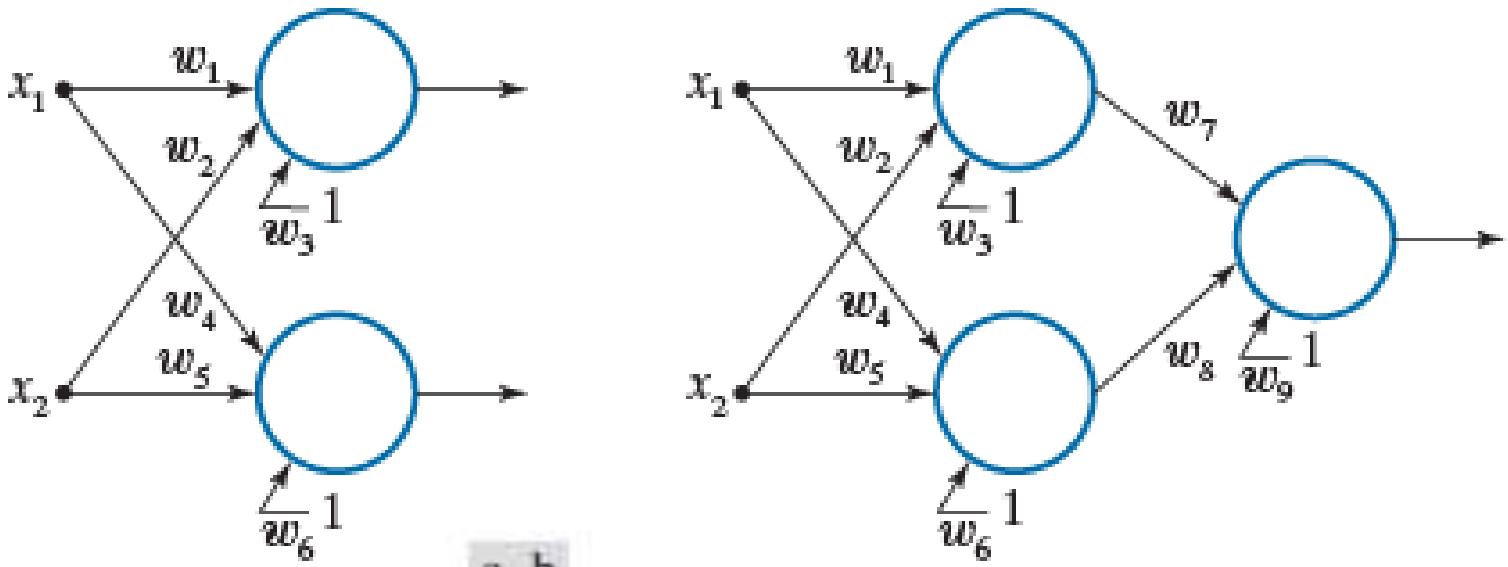


a b c

FIGURE 12.27 The XOR classification problem in 2-D. (a) Truth table definition of the XOR operator. (b) 2-D pattern classes formed by assigning the XOR truth values (1) to one pattern class, and false values (0) to another. The simplest decision boundary between the two classes consists of two straight lines. (c) Nonlinear (quadratic) boundary separating the two classes.

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Limitation of Single Linear Decision Boundary



a b

FIGURE 12.28 (a) Minimum perceptron solution to the XOR problem in 2-D. (b) A solution that implements the XOR truth table in Fig. 12.27(a).

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Multilayer Feedforward Neural Networks

- **Neural networks**
  - interconnected perceptron-like computing elements called artificial neurons
- **Model of an artificial neuron**
  - The output of one unit affects the behavior of all units following it
    - Hard thresholding function is not that good

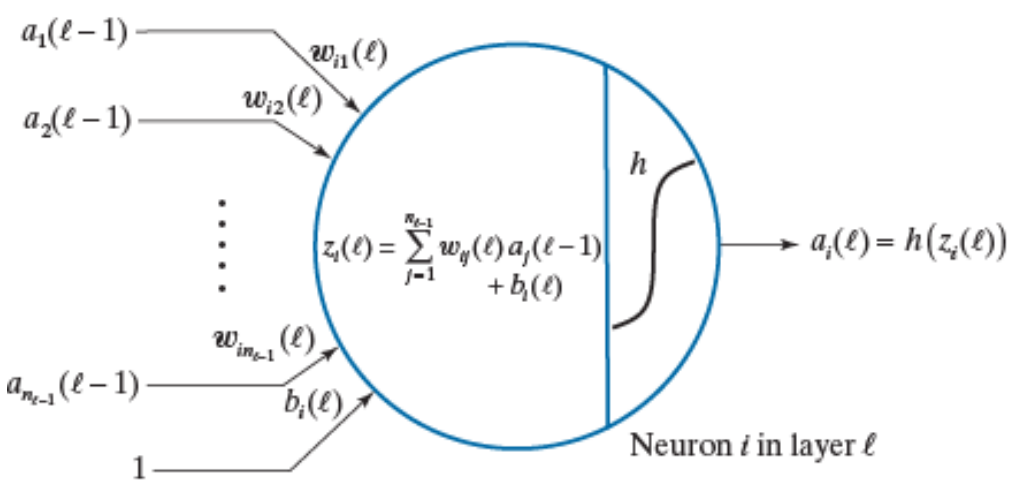
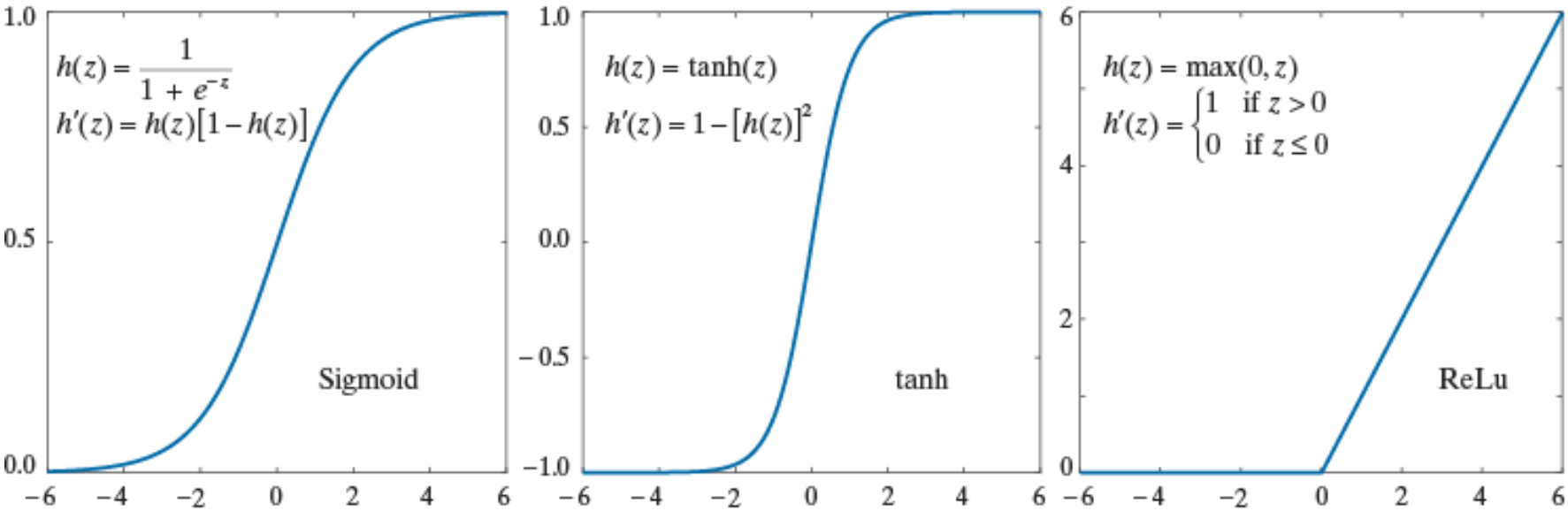


FIGURE 12.29 Model of an artificial neuron, showing all the operations it performs. The "ℓ" is used to denote a particular layer in a layered network.

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Activation Functions

- **The perceptron's sensitivity to the sign of small signals can be problematic**
  - Use a smooth function



a b c

FIGURE 12.30 Various activation functions. (a) Sigmoid. (b) Hyperbolic tangent (also has a sigmoid shape, but it is centered about 0 in both dimensions). (c) Rectifier linear unit (ReLU).

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Fully Connected Neural Network

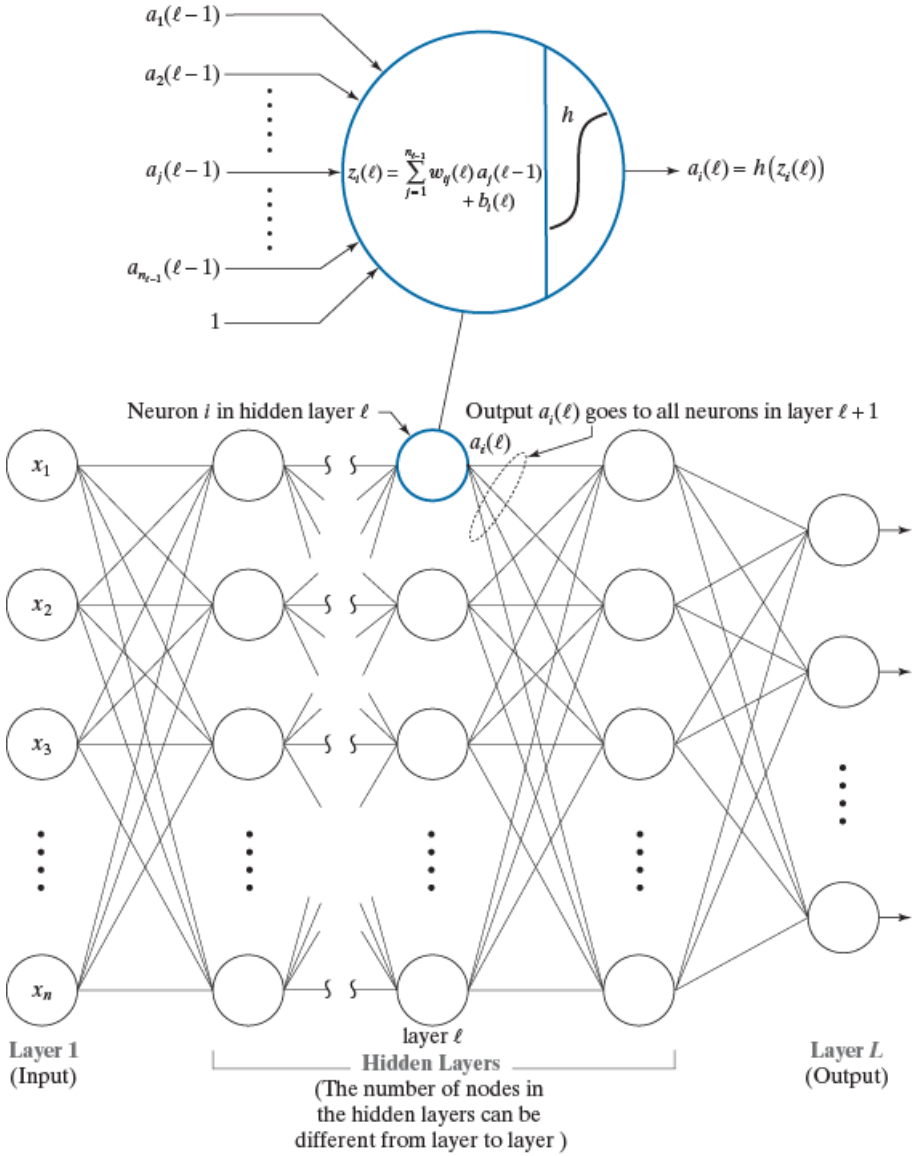


FIGURE 12.31 General model of a feedforward, fully connected neural net. The neuron is the same as in Fig. 12.29. Note how the output of each neuron goes to the input of all neurons in the following layer, hence the name *fully connected* for this type of architecture.

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Fully Connected Neural Network

- **Fully connected neural network**
  - A layer in the network: a set of nodes in a column of the network
  - No loops in the network → feedforward networks
    - Shallow neural network: a single hidden layer
    - Deep neural network: two or more hidden layers

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Forward Pass

- A forward pass maps the input layer to the output layer

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \\ 1 \end{bmatrix}$$

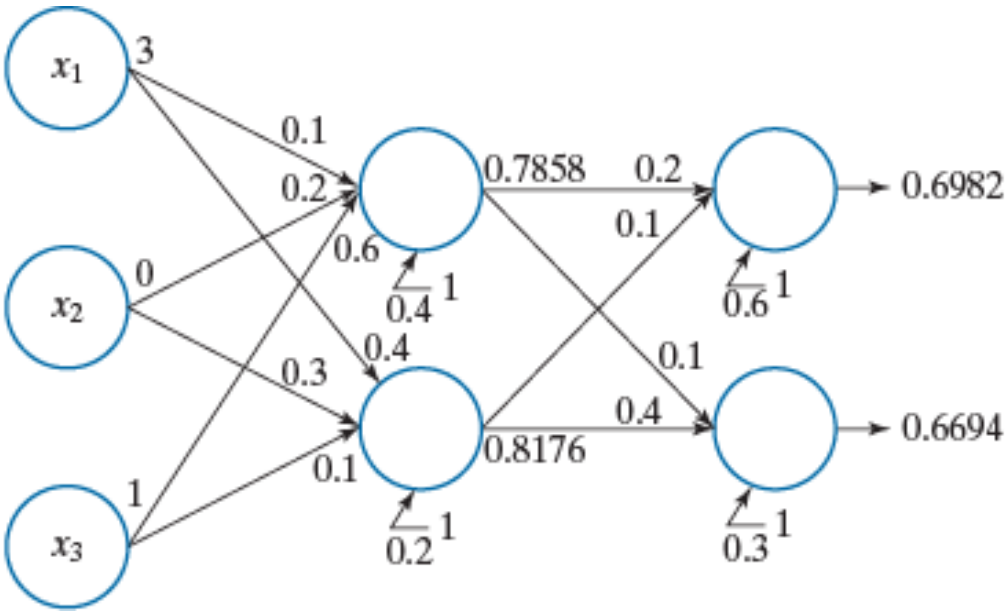


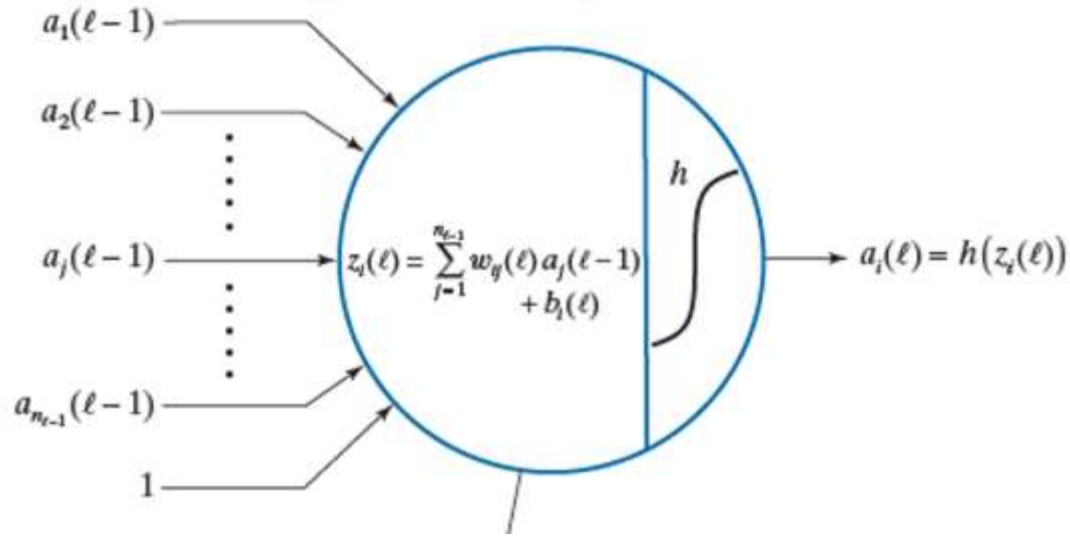
FIGURE 12.32 A small, fully connected, feedforward net with labeled weights, biases, and outputs. The activation function is a sigmoid.

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Matrix Formulation of the Forward Pass

$$\mathbf{a}(1) = \mathbf{x}$$

$$\mathbf{W}(\ell) = \begin{bmatrix} w_{11}(\ell) & w_{12}(\ell) & \cdots & w_{1n_{\ell-1}}(\ell) \\ w_{21}(\ell) & w_{22}(\ell) & \cdots & w_{2n_{\ell-1}}(\ell) \\ \vdots & \vdots & \cdots & \\ w_{n_\ell 1}(\ell) & w_{n_\ell 2}(\ell) & \cdots & w_{n_\ell n_{\ell-1}}(\ell) \end{bmatrix}$$



$$\mathbf{z}(\ell) = \mathbf{W}(\ell)\mathbf{a}(\ell-1) + \mathbf{b}(\ell)$$

$$\mathbf{a}(\ell) = h[\mathbf{z}(\ell)] = \begin{bmatrix} h(z_1(\ell)) \\ h(z_2(\ell)) \\ \vdots \\ h(z_{n_\ell}(\ell)) \end{bmatrix}$$

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Matrix Formulation of the Forward Pass

$$\mathbf{a}(1) = \mathbf{x} = \begin{bmatrix} 3 \\ 0 \\ 1 \end{bmatrix} \quad \mathbf{W}(2) = \begin{bmatrix} 0.1 & 0.2 & 0.6 \\ 0.4 & 0.3 & 0.1 \end{bmatrix} \quad \mathbf{W}(3) = \begin{bmatrix} 0.2 & 0.1 \\ 0.1 & 0.4 \end{bmatrix}$$

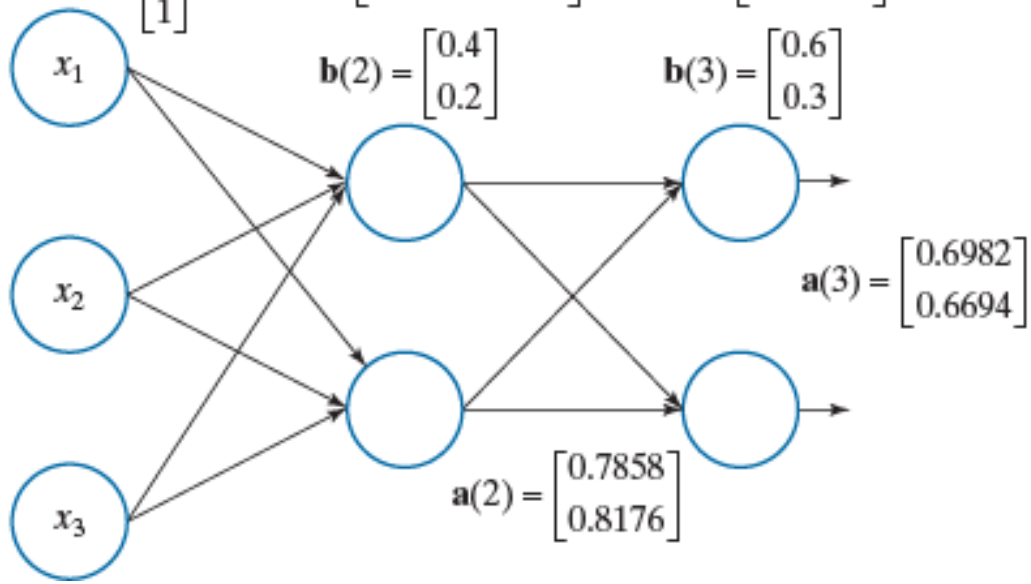


FIGURE 12.33 Same as Fig. 12.32, but using matrix labeling.

$$\mathbf{W}(\ell) = \begin{bmatrix} w_{11}(\ell) & w_{12}(\ell) & \cdots & w_{1n_{\ell-1}}(\ell) \\ w_{21}(\ell) & w_{22}(\ell) & \cdots & w_{2n_{\ell-1}}(\ell) \\ \vdots & \vdots & \cdots & \\ w_{n_\ell 1}(\ell) & w_{n_\ell 2}(\ell) & \cdots & w_{n_\ell n_{\ell-1}}(\ell) \end{bmatrix}$$

$$\mathbf{a}(\ell) = h[\mathbf{z}(\ell)] = \begin{bmatrix} h(z_1(\ell)) \\ h(z_2(\ell)) \\ \vdots \\ h(z_{n_\ell}(\ell)) \end{bmatrix}$$

$$\mathbf{z}(\ell) = \mathbf{W}(\ell)\mathbf{a}(\ell - 1) + \mathbf{b}(\ell)$$

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Matrix Formulation of the Forward Pass for All Patterns

TABLE 13.2
Steps in the matrix computation of a forward pass through a fully connected, feedforward multilayer neural net.

Step	Description	Equations
Step 1	Input patterns	$\mathbf{A}(1) = \mathbf{X}$
Step 2	Feedforward	For $\ell = 2, \ldots, L$, compute $\mathbf{Z}(\ell) = \mathbf{W}(\ell)\mathbf{A}(\ell - 1) + \mathbf{B}(\ell)$ and $\mathbf{A}(\ell) = h(\mathbf{Z}(\ell))$
Step 3	Output	$\mathbf{A}(L) = h(\mathbf{Z}(L))$

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Backpropagation to Train Deep Neural Network

- **A neural network is defined completely by its weights, biases, activation function**
- **Training a neural network refers to using training patterns to estimate these parameters**
  - Input the pattern vectors
  - A forward pass through the network to classify all the patterns of the training set and determine the error
  - A backward pass that feeds the output error back through the network to compute the changes required to update the parameters
  - Update the parameters

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Backpropagation

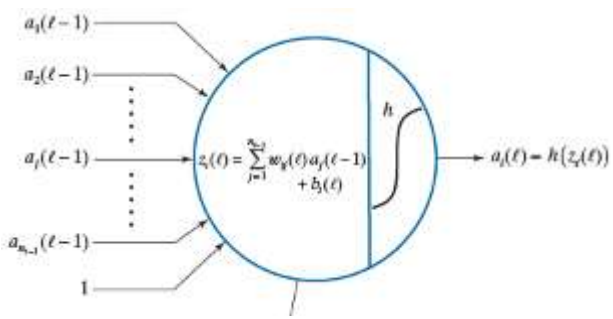
- **To find the parameters that minimize an error function**
- **The total network output error of all the output nodes over all training patterns**
  - Error at a single output node

$$E_j = \frac{1}{2}\left(r_j - a_j(L)\right)^2$$

$$E = \sum_{j=1}^{n_L} E_j = \frac{1}{2}\sum_{j=1}^{n_L}\left(r_j - a_j(L)\right)^2 = \frac{1}{2}\left\|\mathbf{r} - \mathbf{a}(L)\right\|^2$$

  - We need to know how E changes w.r.t. the weights

$$\delta_j(L) = \frac{\partial E}{\partial z_j(L)}$$



為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

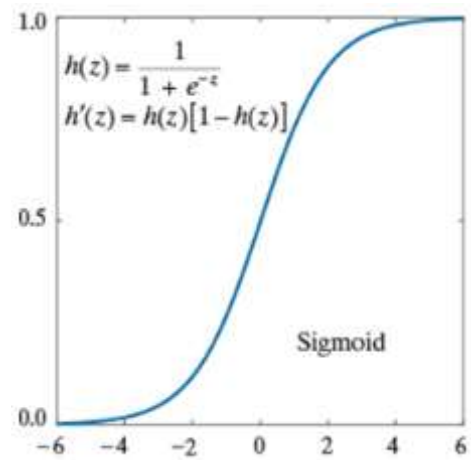
Backpropagation

- **To know how E changes w.r.t. the weights**

$$\delta_j(L) = \frac{\partial E}{\partial z_j(L)}$$

$$\delta_j(L) = \frac{\partial E}{\partial z_j(L)} = \frac{\partial E}{\partial a_j(L)} \frac{\partial a_j(L)}{\partial z_j(L)} = \frac{\partial E}{\partial a_j(L)} \frac{\partial h\left(z_j(L)\right)}{\partial z_j(L)}$$

$$= \frac{\partial E}{\partial a_j(L)} h'\left(z_j(L)\right)$$

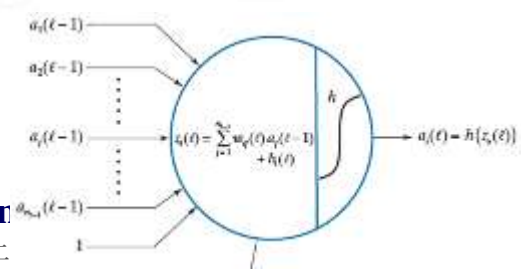


- If using sigmoid

$$E = \sum_{j=1}^{n_L} E_j = \frac{1}{2} \sum_{j=1}^{n_L} \left(r_j - a_j(L)\right)^2$$

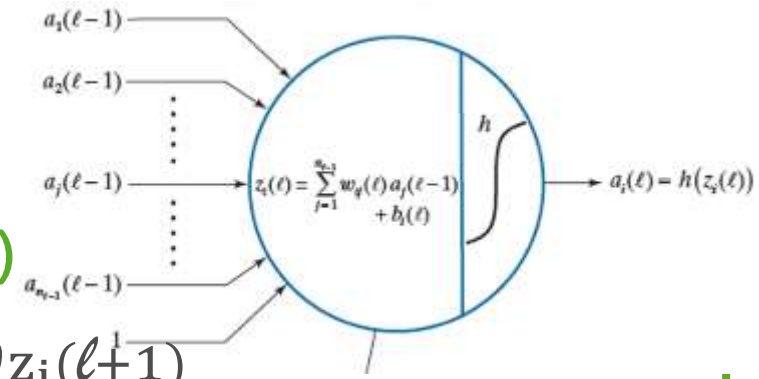
$$= \frac{1}{2} \| \mathbf{r} - \mathbf{a}(L) \|^2$$

$$\delta_j(L) = h\left(z_j(L)\right)\left[1 - h\left(z_j(L)\right)\right]\left[a_j(L) - r_j\right]$$



為保護所參考來源之智慧財產權，此教材僅為上

Backpropagation



- **To find $\delta_j(\ell)$ in terms of $\delta_j(\ell+1)$**

$$\delta_j(\ell) = \frac{\partial E}{\partial z_j(\ell)} = \sum_i \frac{\partial E}{\partial z_i(\ell+1)} \frac{\partial z_i(\ell+1)}{\partial z_j(\ell)}$$

$$= \sum_i \frac{\partial E}{\partial z_i(\ell+1)} \frac{\partial z_i(\ell+1)}{\partial a_j(\ell)} \frac{\partial a_j(\ell)}{\partial z_j(\ell)}$$

$$= \sum_i \delta_i(\ell+1) \frac{\partial z_i(\ell+1)}{\partial a_j(\ell)} h'(z_j(\ell))$$

$$= h'(z_j(\ell)) \sum_i w_{ij}(\ell+1)\, \delta_i(\ell+1)$$

- **To find $w_{ij}(\ell)$**

$$\frac{\partial E}{\partial w_{ij}(\ell)} = \frac{\partial E}{\partial z_i(\ell)} \frac{\partial z_i(\ell)}{\partial w_{ij}(\ell)} = \delta_i(\ell) \frac{\partial z_i(\ell)}{\partial w_{ij}(\ell)} = a_j(\ell-1)\, \delta_i(\ell)$$

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Backpropagation

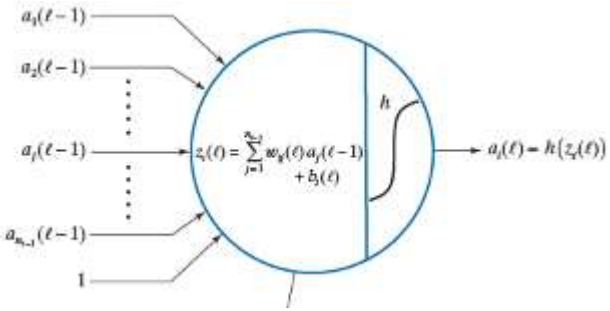
- **To find** bias

$$\frac{\partial E}{\partial b_i(\ell)} = \delta_i(\ell)$$

- **To update parameters using gradient descent**

$$\begin{aligned} w_{ij}(\ell) &= w_{ij}(\ell) - \alpha \frac{\partial E(\ell)}{\partial w_{ij}(\ell)} \\ &= w_{ij}(\ell) - \alpha \delta_i(\ell) a_j(\ell - 1) \end{aligned}$$

$$\begin{aligned} b_i(\ell) &= b_i(\ell) - \alpha \frac{\partial E}{\partial b_i(\ell)} \\ &= b_i(\ell) - \alpha \delta_i(\ell) \end{aligned}$$



為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。

Efficient BackProp

- From LeCun et al, "Neural Networks: tricks of the trade," Springer, 1998.

1 Introduction

Backpropagation is a very popular neural network learning algorithm because it is conceptually simple, computationally efficient, and because it often works. However, getting it to work well, and sometimes to work at all, can seem more of an art than a science.

為保護所參考來源之智慧財產權，此教材僅為上課時教學之用。